CHƯƠNG 4

Thiết kế giao diện và trải nghiệm người dùng với AI tạo sinh

[Ứng dụng Trí tuệ nhân tạo]



[Nguyễn Tuấn Anh]
• [Thái Nguyên, 2026]

Giới thiệu nội dung

Mục tiêu chương

- Mô tả được vai trò của UI/UX trong ứng dụng AI tạo sinh, hệ sinh thái công cụ từ tạo mẫu nhanh đến sản xuất, và kiến trúc 4 tầng của chatbot GenAI.
- Phân tích được lợi ích (tốc độ, chất lượng, hạ tầng đám mây) và thách thức (sai lệch, chi phí, độ tin cậy, đạo đức) khi tích hợp AI vào thiết kế UI/UX.
- Vận dụng được Well-Architected

Nội dung chính

1. Ứng dụng AI trong UI/UX: Tổng quan và xu hướng
2. Phân tích ưu nhược điểm của AI trong thiết kế UI/UX
3. Thiết kế giao diện người dùng bằng AI
4. Thực hành: Xây dựng prototype với AI tạo sinh

Khởi động: AI cần giao diện để đến với người dùng

- Hầu hết người dùng cuối **không phải là lập trình viên** – họ không tương tác với AI qua terminal hay API.
- Họ cần một **giao diện trực quan**: nhập câu hỏi, kéo thả tệp, xem kết quả và nhận phản hồi theo thời gian thực.
- UI/UX là **cầu nối không thể thiếu** giữa sức mạnh của AI và nhu cầu thực tiễn của con người.

Mối quan hệ hai chiều:

- AI giúp thiết kế UI/UX tốt hơn
- UI/UX tốt giúp khai thác AI hiệu quả hơn

Điểm cốt lõi

Một giao diện kém có thể khiến ngay cả mô hình AI mạnh nhất cũng trở nên **vô dụng** – vì người dùng không biết cách tương tác với nó.

Ba trụ cột của ứng dụng AI tạo sinh

Mọi ứng dụng AI tạo sinh thành công đều đứng trên ba trụ cột, trong đó **UI/UX đóng vai trò trung tâm**:

1. **Chuyển đổi trải nghiệm người dùng** (UX Transformation)
   - Cá nhân hóa giao diện theo hành vi và sở thích từng người dùng.
   - Tạo ra tương tác thích ứng – hệ thống "học" cách phục vụ từng cá nhân tốt hơn theo thời gian.
2. **Nâng cao năng suất lao động** (Productivity Enhancement)
   - Tự động hóa tác vụ lặp đi lặp lại $\rightarrow$ con người tập trung vào sáng tạo và ra quyết định.
3. **Hiệu quả vận hành** (Operational Efficiency)
   - Tối ưu hóa quy trình nghiệp vụ thông qua phân tích thông minh và dự đoán.

Hệ sinh thái công cụ xây dựng giao diện cho ứng dụng AI

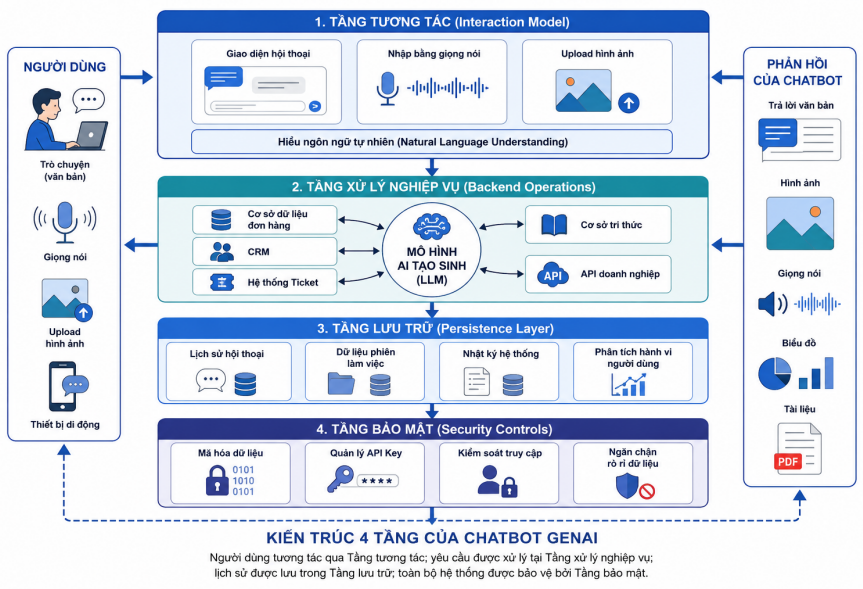
Cấp độ	Công cụ	Thế mạnh chính	Hạn chế chính
Tạo mẫu nhanh	Gradio, Streamlit	Dựng giao diện trong vài phút; ít code; tích hợp sẵn ML/AI	Không phù hợp cho sản quy mô lớn; tùy biến hạn
Sản xuất	Flask, Django, Node.js + React/Vue	Bảo mật; khả năng mở rộng; tích hợp doanh nghiệp	Yêu cầu kỹ năng front-e backend; thời gian phát dài hơn
Tích hợp front-end AI	LangChain.js, Shiny	Gắn LLM trực tiếp vào web; reactive programming	Phụ thuộc hệ sinh thái còn non trẻ

Nguyên tắc chọn công cụ

Gradio = "bọc" mô hình AI thành web app | **Streamlit** = dashboard & data apps | **Flask/Django** = production systems

Kiến trúc 4 tầng của Chatbot GenAI

1. **Tầng tương tác** (Interaction Model)
   - Giao diện chat, input voice, upload ảnh.
   - Quyết định ấn tượng đầu tiên và mức độ dễ sử dụng.
2. **Tầng xử lý nghiệp vụ** (Backend Operations)
   - Tích hợp CRM, cơ sở dữ liệu, hệ thống quản lý ticket.
   - Cho phép chatbot thực hiện tác vụ thực tế.
3. **Tầng lưu trữ** (Persistence Layer)
   - Lịch sử hội thoại, dữ liệu phiên – phân tích và cải



Bảo mật xuyên suốt cả 4 tầng, không chỉ là một lớp riêng biệt.

Thiết kế lấy người dùng làm trung tâm trong kỷ nguyên AI

- **AI phải dễ sử dụng với mọi đối tượng** – bao gồm cả những người không rành công nghệ.
- **Hướng dẫn sử dụng rõ ràng** là yếu tố thường bị bỏ qua nhưng có tác động lớn:
  - Menu thả xuống chứa prompt mẫu.
  - Tooltip giải thích chức năng.
  - Gợi ý thời gian thực khi người dùng đang nhập câu hỏi.

Công cụ phân tích hành vi:

- Mixpanel, Hotjar – theo dõi cách người dùng tương tác với AI.
- Google Optimize – A/B testing phiên bản giao diện.
- Figma – phác thảo và kiểm thử ý tưởng nhanh.

Ví dụ thực tế

Công cụ tạo CV dùng AI: menu thả xuống chọn ngành nghề $\rightarrow$ tự động đề xuất cấu trúc truy vấn phù hợp – thay vì để người dùng loay hoay.

Xu hướng: Giao diện thích ứng và đa phương thức

1. **Giao diện thích ứng theo ngữ cảnh** (Context-Aware Interface)
   - Giao diện tự động điều chỉnh bố cục, nội dung dựa trên hành vi và lịch sử người dùng.
   - Người dùng mới $\rightarrow$ hướng dẫn chi tiết + prompt gợi ý.
   - Người dùng quen $\rightarrow$ giao diện thu gọn, tối đa không gian làm việc.
2. **Giao diện đa phương thức** (Multimodal Interface)
   - Tương tác qua **nhiều kênh đồng thời**: văn bản, hình ảnh, giọng nói.
   - Ví dụ: Chụp ảnh bản vẽ tay $\rightarrow$ AI chuyển thành sơ đồ kỹ thuật số $\rightarrow$ chỉnh sửa bằng lời nói – tất cả trong một phiên.

Đòi hỏi phối hợp nhiều mô hình AI (thị giác, ngôn ngữ, giọng nói) và đặt ra thách thức mới về thiết kế tương tác.

Tổng kết Section 1 – Tự kiểm tra

Những điểm cần nhớ

1. **UI/UX là trụ cột trung tâm** quyết định thành công của ứng dụng AI tạo sinh – không chỉ là "vỏ bọc" mà là cầu nối giữa sức mạnh kỹ thuật và nhu cầu người dùng.
2. **Hệ sinh thái công cụ** trải từ tạo mẫu nhanh (Gradio, Streamlit) đến sản xuất (Flask, Django) và tích hợp front-end AI (LangChain.js) – mỗi cấp độ phục vụ một giai đoạn khác nhau.
3. **Kiến trúc chatbot 4 tầng**: Tương tác – Nghiệp vụ – Lưu trữ – Bảo mật. Bảo mật phải xuyên suốt, không phải lớp riêng.
4. **Xu hướng**: Giao diện thích ứng theo ngữ cảnh và đa phương thức đang định hình tương lai UI/UX cho AI.

Tự kiểm tra nhanh:

1. Khi nào nên dùng Gradio thay vì Flask để xây dựng giao diện AI?

Lợi ích 1: Tăng tốc quy trình phát triển

- AI tự động hóa việc tạo code khởi tạo (boilerplate), sinh API tích hợp, và đề xuất cấu trúc giao diện từ mô tả ngôn ngữ tự nhiên.

Con số thực tế từ các tổ chức lớn:

- **Google**: Hơn **25%** mã nguồn mới do AI tạo ra.
- **Amazon**: Di chuyển hàng chục nghìn ứng dụng Java với Amazon Q Developer, tiết kiệm **4.500 năm công** và **260 triệu USD/năm**.
- **Meta**: Giảm hơn **20%** mã trùng lặp với CodeCompose.
- **Microsoft**: Tốc độ viết code tăng **30–40%** với GitHub Copilot.

Gradio và Streamlit cho phép dựng giao diện demo cho mô hình AI trong **vài phút** – công việc trước đây cần front-end developer mất **vài giờ đến vài ngày**.

Lợi ích 2: Nâng cao chất lượng sản phẩm

- **Gợi ý thời gian thực** dựa trên ngữ cảnh – phát hiện lỗi tiềm ẩn trước khi commit.
- **Tự động kiểm tra tính nhất quán**: màu sắc, khoảng cách, typography – những lỗi nhỏ nhưng dễ mắc khi làm thủ công.
- **Tự động sinh ca kiểm thử** (test case) bao phủ nhiều kịch bản người dùng.
- **Tạo tài liệu kỹ thuật thông minh** (automated documentation) luôn đồng bộ với code.
- **Thu hẹp khoảng cách** giữa đội ngũ kỹ thuật và phi kỹ thuật:
  - PM mô tả yêu cầu bằng ngôn ngữ tự nhiên $\rightarrow$ AI chuyển thành code khung.

AI không chỉ viết code **nhanh hơn** – còn viết code **tốt hơn**: phát hiện lỗi sớm, đề xuất refactoring, tự động kiểm thử.

Lợi ích 3: Sức mạnh của hạ tầng đám mây

Các nền tảng đám mây đóng vai trò **chất xúc tác**, khuếch đại lợi ích của AI trong UI/UX:

- **Pay-per-use**: Đội phát triển nhỏ tiếp cận được mô hình AI tiên tiến nhất (GPT-4o, Claude, Gemini) mà không cần đầu tư hạ tầng ban đầu.
- **Auto-scaling**: Hệ thống không bị quá tải khi lượng người dùng tăng đột biến.
- **Dịch vụ quản lý**: Đơn giản hóa triển khai, giám sát và bảo trì.
- **Chứng nhận bảo mật và tuân thủ**: GDPR, HIPAA, ISO, SOC 2 được nhà cung cấp duy trì – giảm gánh nặng pháp lý cho đội phát triển.

Đám mây biến AI từ “đặc quyền của ông lớn công nghệ” thành **công cụ cho mọi đội phát triển**.

Thách thức 1: Bài toán sai lệch (Bias)

Sai lệch là một trong những thách thức **nghiêm trọng nhất** của AI tạo sinh, ảnh hưởng trực tiếp đến UI/UX:

- **Sai lệch kiến trúc mô hình** (Model Architecture Bias): Số lớp, loại kết nối, hàm kích hoạt $\rightarrow$ hiệu suất không đồng đều giữa các ngôn ngữ/lĩnh vực.
- **Sai lệch hàm mục tiêu** (Objective Function Bias): Tối ưu theo tỷ lệ nhấp chuột $\rightarrow$ AI ưu tiên tiêu đề "giật gân" thay vì chính xác.
- **Sai lệch chuyên môn hóa** (Task/Domain Specialization): Mô hình tinh chỉnh cho tài chính có thể thiếu công bằng khi áp dụng sang lĩnh vực khác.
- **Sai lệch dữ liệu huấn luyện**: Dữ liệu từ các nước phát triển $\rightarrow$ thiết kế không phù hợp văn hóa cho khu vực khác.

Lưu ý

Hệ thống gợi ý giao diện huấn luyện trên dữ liệu thiên lệch có thể tạo ra thiết kế **phân biệt đối xử** mà đội phát triển không nhận ra.

Thách thức 2: Chi phí vận hành

- Khác với phần mềm truyền thống – nơi chi phí chủ yếu đến từ phát triển ban đầu và bảo trì – ứng dụng AI tạo sinh còn phát sinh **chi phí inference liên tục** cho mỗi yêu cầu.
- Một chatbot GenAI phục vụ hàng nghìn người dùng đồng thời đòi hỏi tài nguyên tính toán lớn cho inference thời gian thực **và** huấn luyện lại định kỳ.

Chiến lược tối ưu chi phí:

- Chọn mô hình nhỏ hơn cho tác vụ đơn giản.
- Xử lý theo batch thay vì từng yêu cầu.
- Caching kết quả cho truy vấn tương tự.

Đánh đổi

- Mô hình nhỏ $\rightarrow$ kém chính xác
- Batch $\rightarrow$ tăng độ trễ
- Caching $\rightarrow$ kết quả lỗi thời

Thách thức 3: Chất lượng đầu ra và độ tin cậy

- Các benchmark như MMLU (57 môn học), HELM, Artificial Analysis cho thấy LLM vẫn **gặp khó khăn với suy luận logic phức tạp** mà con người thấy hiển nhiên.
- **Hallucination** – AI tự tin tạo ra thông tin không có thật – là rủi ro đặc thù của GenAI mà thiết kế giao diện truyền thống không phải đối mặt.

Đặc biệt nhạy cảm trong UI/UX

Một giao diện **đẹp mắt nhưng đưa ra gợi ý sai** có thể khiến người dùng tin tưởng và làm theo $\rightarrow$ hậu quả nghiêm trọng trong **y tế**, **tài chính**, **pháp lý**.

- HELM và Artificial Analysis còn đánh giá cả **sai lệch**, **thông tin sai lệch** và **rủi ro đạo đức** – những yếu tố tác động trực tiếp đến UX.

Thách thức 4: Vai trò của con người trong kỷ nguyên AI

Liệu AI có thay thế được nhà thiết kế con người?

- **AI là "companion" và "enabler"**, không phải "replacer".
- AI xuất sắc trong:
  - Tự động hóa tác vụ lặp đi lặp lại.
  - Đề xuất phương án thiết kế.
- Con người **không thể thay thế** trong:
  - Xác định luồng người dùng và kiến trúc thông tin.
  - Lựa chọn chiến lược thiết kế.
  - Đánh giá tác động đạo đức của mẫu tương tác.

AI đề xuất – Con người quyết định. Các quyết định thiết kế mang tính chiến lược vẫn đòi hỏi sự phán đoán của con người.

Cân bằng giữa lợi ích và thách thức

Ba nguyên tắc để tận dụng sức mạnh của AI trong UI/UX mà không rơi vào cạm bẫy:

1. **Đánh giá dựa trên ngữ cảnh cụ thể**
   - Một công cụ AI hiệu quả cho tạo mẫu nhanh có thể không phù hợp cho thiết kế hệ thống y tế yêu cầu độ chính xác tuyệt đối.
   - **Không có giải pháp "một cho tất cả".**
2. **Luôn duy trì vòng kiểm soát của con người** (Human-in-the-Loop)
   - AI đề xuất – Con người quyết định.
3. **Đo lường và cải tiến liên tục**
   - Dùng CSAT (tỷ lệ hài lòng), Interaction Rate, Bounce Rate để đánh giá khách quan.

Tổng kết Section 2 – Tự kiểm tra

Những điểm cần nhớ

1. **AI tăng tốc phát triển** (Google 25%+, Amazon 4.500 năm công) và **nâng cao chất lượng** (gợi ý thời gian thực, tự động kiểm thử, sinh tài liệu).
2. **Hạ tầng đám mây** (pay-per-use, auto-scaling) giúp mọi đội phát triển tiếp cận được mô hình AI tiên tiến.
3. **Bốn nhóm thách thức chính**: sai lệch (kiến trúc, hàm mục tiêu, chuyên môn hóa, dữ liệu), chi phí inference, hallucination và độ tin cậy, câu hỏi về vai trò con người.
4. **Ba nguyên tắc cân bằng**: đánh giá theo ngữ cảnh, human-in-the-loop, đo lường và cải tiến liên tục.

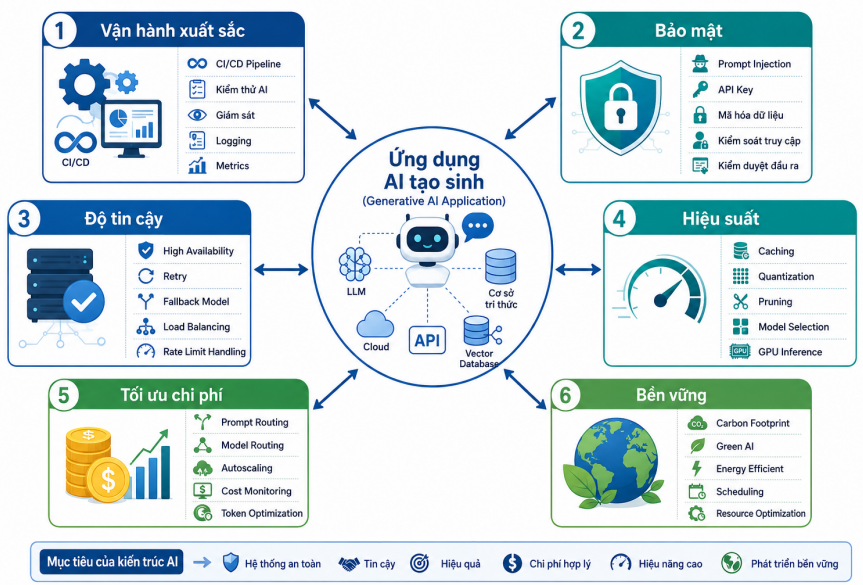
Tự kiểm tra nhanh:

1. Kể tên 3 loại sai lệch trong AI và giải thích mỗi loại ảnh hưởng đến

Well-Architected Framework cho ứng dụng AI

6 trụ cột – mỗi trụ cột có ý nghĩa đặc thù khi áp dụng cho GenAI:

1. **Vận hành xuất sắc**: CI/CD pipeline tự động kiểm thử chất lượng đầu ra mô hình.
2. **Bảo mật**: Chống prompt injection, bảo vệ API key, ngăn rò rỉ dữ liệu huấn luyện.
3. **Độ tin cậy**: Xử lý quá tải nhẹ nhàng – trả về phản hồi rút gọn thay vì crash.
4. **Hiệu suất**: Chọn mô hình phù hợp, caching, pruning, quantization.
5. **Tối ưu chi phí**: Prompt



Well-Architected Framework: "checklist" toàn diện khi thiết kế hệ thống AI.

Thiết kế bảo mật và quyền riêng tư ngay từ đầu

- **Bảo mật dữ liệu theo thiết kế** (Data Privacy by Design):
  - Mã hóa khi truyền tải (TLS) và khi lưu trữ (AES-256).
  - **Differential Privacy**: thêm "nhiễu" có kiểm soát để không thể truy ngược về cá nhân cụ thể.
- **Kiểm soát truy cập theo vai trò** (RBAC):
  - Người dùng chỉ xem và thao tác với những gì họ được phép.

Ví dụ: Chatbot pháp lý

- **Luật sư**: xem toàn bộ lịch sử hội thoại và tài liệu tham khảo.
- **Khách hàng**: chỉ thấy phần trả lời cuối cùng.

Lưu ý

Bảo mật không phải là thứ "hàn gắn" sau khi giao diện đã hoàn thiện – phải được thiết kế **ngay từ đầu**.

Giải thích được quyết định của AI

- **Tính "hộp đen"** là rào cản lớn nhất với việc chấp nhận AI trong lĩnh vực nhạy cảm.
- **Khả năng giải thích mô hình** (Model Interpretability) cung cấp cái nhìn vào quá trình ra quyết định.

Ba công cụ chính:

- **SHAP** (SHapley Additive exPlanations): giải thích tại sao khoản vay bị từ chối (tài chính).
- **LIME** (Local Interpretable Model-agnostic Explanations): chỉ ra vùng ảnh X-quang khiến AI nghi ngờ khối u (y tế).
- **Google Explainable AI Toolkit**: giải thích quyết định pháp lý trước tòa.

Trong UI, giải thích được trình bày dưới dạng **explainability dashboard** hoặc chú thích trực quan – tô màu vùng ảnh quan trọng, hiển thị biểu đồ mức độ đóng góp của từng đặc trưng.

Thiết kế giao diện hội thoại: Từ prompt đến sản phẩm

- **System Prompt**: định nghĩa "tính cách", giọng điệu và ranh giới hành vi của AI xuyên suốt mọi tương tác.
  - Ví dụ: "Bạn là trợ lý pháp lý, chỉ trả lời dựa trên luật hiện hành, không đưa ra lời khuyên cá nhân."
- **User Prompt**: câu truy vấn cụ thể từ người dùng.

Kiến trúc kỹ thuật:

- **API Gateway**: điểm vào duy nhất – xác thực, định tuyến request.
- Tầng tương tác với mô hình nền tảng (Bedrock, OpenAI API).
- **Lambda Functions** / serverless: xử lý logic nghiệp vụ.
- Tích hợp hệ thống ngoài: CRM, database, thanh toán.
- Giám sát: CloudWatch + vòng phản hồi cải tiến liên tục.

Lưu ý

System prompt phải được **bảo vệ** khỏi bị lộ hoặc bị ghi đè bởi người dùng.

Guardrails: Rào chắn bảo vệ đa lớp

- Guardrails hoạt động như "bộ lọc" nhiều lớp cho đầu ra của AI:

1. **Kiểm tra tính xác thực**:
   - Đánh giá của con người (human evaluation).
   - **LLM as a Judge**: dùng một LLM khác làm "giám khảo" đánh giá phản hồi.
2. **Ngăn chặn rò rỉ thông tin**:
   - Không chia sẻ thông tin bảo mật.
   - Không đưa ra cam kết trái phép thay mặt tổ chức.
3. **Lọc nội dung độc hại**:
   - Phát hiện và chặn trước khi hiển thị cho người dùng.

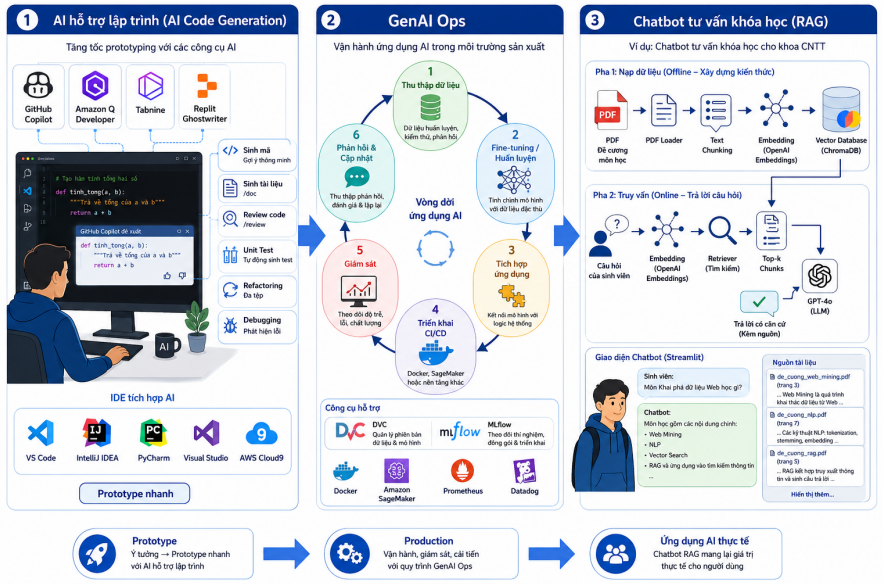
Không nên chỉ dùng **một** cơ chế duy nhất – kết hợp nhiều lớp guardrails để tăng độ an toàn.

RAG: Sinh nội dung tăng cường truy xuất

Vấn đề: LLM có kiến thức "đóng băng" tại thời điểm huấn luyện.
Giải pháp RAG:

1. Nhận câu hỏi từ người dùng.
2. Chuyển câu hỏi thành **vector embedding**.
3. Tìm kiếm trong **vector database** những đoạn tài liệu liên quan nhất.
4. Ghép các đoạn tìm được vào prompt.
5. LLM sinh câu trả lời có **căn cứ**.

Lợi ích:



Pipeline RAG: nạp dữ liệu (trên) và truy vấn (dưới).

Công cụ thiết kế UI được AI hỗ trợ

Công cụ thiết kế (UI Design Tools):

- **Uizard**: Chuyển bản phác thảo tay thành wireframe kỹ thuật số; sinh bố cục từ mô tả văn bản.
- **Figma AI**: Sinh nội dung mẫu, đề xuất biến thể màu sắc và typography, tự động đặt tên layer.

→ Tạo ra **bản vẽ, bố cục, nguyên mẫu tĩnh**.

Công cụ hiện thực hóa (UI Implementation Tools):

- **Gradio, Streamlit**: Dựng giao diện web từ Python trong vài phút.
- **Flask, Django + React/Vue**: Xây dựng ứng dụng sản xuất hoàn chỉnh.

→ Biến thiết kế thành **code chạy được**.

Quy trình điển hình

Vòng lặp giám sát – cải tiến – lặp lại

Thiết kế UI cho ứng dụng AI không phải là hoạt động "làm một lần rồi xong"

1. **Triển khai** (Deploy): Đưa giao diện và AI vào vận hành.
2. **Giám sát** (Monitor): Theo dõi hành vi người dùng thực tế qua Mixpanel, Hotjar.
3. **Xác định điểm ma sát** (Identify Friction): Phát hiện nơi người dùng gặp khó khăn hoặc rời bỏ.
4. **Cải tiến** (Improve): Điều chỉnh giao diện hoặc prompt dựa trên dữ liệu.
5. **Triển khai lại** (Re-deploy): Lặp lại chu trình.

Các công cụ như Mixpanel, Hotjar, Google Optimize là "tai mắt" giúp đội phát triển **hiểu điều gì đang thực sự diễn ra**, thay vì chỉ dựa vào trực giác.

Tổng kết Section 3 – Tự kiểm tra

Những điểm cần nhớ

1. **Well-Architected Framework 6 trụ cột** cung cấp "checklist" toàn diện khi thiết kế hệ thống AI – từ vận hành, bảo mật, độ tin cậy đến hiệu suất, chi phí và bền vững.
2. **Bảo mật phải được thiết kế ngay từ đầu**: mã hóa (TLS/AES-256), differential privacy, RBAC. Không "hàn gắn" sau khi giao diện đã xong.
3. **Guardrails và RAG là "tính năng bắt buộc"**: Guardrails lọc và kiểm soát đầu ra; RAG tách kiến thức tĩnh khỏi tương tác thời gian thực qua embedding + vector database.
4. **Phân biệt công cụ thiết kế** (Uizard, Figma AI) và **công cụ hiện thực hóa** (Gradio, Flask) – hai giai đoạn khác nhau, cần phối hợp nhịp nhàng.

Từ ý tưởng đến prototype: Chọn công cụ phù hợp

"Mình đang ở giai đoạn nào?" – Câu trả lời quyết định công cụ nên dùng

Giai đoạn thử nghiệm ý tưởng

Dùng Gradio hoặc Streamlit:

- "Bọc" hàm Python thành web app chỉ với vài dòng code.
- **Gradio**: mạnh với AI tasks – phân loại ảnh, sinh văn bản, nhận diện giọng nói.
- **Streamlit**: mạnh với

Giai đoạn sản xuất

Dùng Flask/Django + React/Vue:

- MVP bằng Gradio/Streamlit.
- Thử nghiệm với người dùng thật.
- Khi ổn định $\rightarrow$ viết lại bằng Flask/Django.
- Đầy đủ: xác thực, bảo

Quy trình 10 bước xây dựng chatbot GenAI (1/2)

1. **Xác định use case**: Vấn đề có thực sự cần chatbot không? Form tĩnh hoặc FAQ có khi hiệu quả và rẻ hơn.
2. **Làm rõ mục đích và tiêu chí thành công**: Chatbot phục vụ ai? Trả lời loại câu hỏi nào? Thế nào là "thành công"?
3. **Chọn nền tảng và nhà cung cấp LLM**: OpenAI (GPT-4o), Anthropic (Claude), Google (Gemini), hoặc mã nguồn mở (Llama, DeepSeek).
4. **Thiết kế kiến trúc chatbot**: 4 tầng – Interaction Model, Backend Operations, Persistence Layer, Security Controls.
5. **Thiết lập kết nối API/SDK**: Viết script Python đơn giản gọi API và kiểm tra phản hồi **trước** khi xây dựng toàn bộ ứng dụng.

Bước 1 và 2 thường bị bỏ qua nhất – nhưng là nền tảng quyết định thành công.

Quy trình 10 bước xây dựng chatbot GenAI (2/2)

6. **Định nghĩa chiến lược quản lý prompt**: System prompt (tính cách, ranh giới) + cấu trúc user prompt rõ ràng (XML tags, few-shot examples).
7. **Thêm guardrails**: Lọc nội dung độc hại, phát hiện câu trả lời không liên quan, giới hạn phạm vi trả lời. Kỹ thuật LLM as a Judge.
8. **Xây dựng cơ sở tri thức RAG**: Chunk tài liệu $\rightarrow$ embedding $\rightarrow$ vector database $\rightarrow$ truy xuất khi có câu hỏi. Quản lý ngữ cảnh hội thoại.
9. **Triển khai prompt và giao diện**: Dùng Streamlit dựng giao diện test nhanh với ô nhập, lịch sử hội thoại, panel nguồn tài liệu.
10. **Giám sát, cải tiến, lặp lại**: Theo dõi CSAT, Interaction Rate, Fallback Rate, Bounce Rate, Self-service Rate.

Chỉ số và hành động

Fallback Rate cao $\rightarrow$ điều chỉnh prompt hoặc bổ sung RAG. | Bounce Rate cao $\rightarrow$ cải thiện luồng hội thoại mở đầu. | CSAT thấp $\rightarrow$ phân

Công cụ tạo code bằng AI: Trợ thủ cho prototyping

Công cụ	Đặc điểm	Phù hợp cho
GitHub Copilot	"Pair programmer" AI trong IDE; gợi ý code từ comment và ngữ cảnh	Viết code hàng ngày, sinh boilerplat nhanh
Amazon Q Developer	Trợ lý toàn diện: sinh code, tài liệu (/doc), review (/review), test, refactoring đa file, khởi tạo dự án (/dev)	Dự án phức tạp, cần AI tham gia sâ vào toàn bộ quy trình
Tabnine	Mô hình local – chạy trên máy, không gửi code lên cloud	Dự án yêu cầu bảo mật dữ liệu nghiê ngặt
Replit Ghostwriter	Gợi ý code, phát hiện lỗi, debugging trong IDE đám mây	Người mới bắt đầu, máy cấu hình thấ

Các IDE hiện đại (VS Code, JetBrains, Visual Studio) đều đã tích hợp AI – **AI đã trở thành một phần không thể thiếu trong môi trường phát triển**.

GenAI Ops: Từ prototype đến production

Giữa prototype và production là một khoảng cách lớn – GenAI Ops là cây cầu bắc qua khoảng cách đó

6 giai đoạn của GenAI Ops:

1. **Thu thập và tiền xử lý dữ liệu**: dữ liệu huấn luyện, kiểm thử, phản hồi người dùng.
2. **Huấn luyện hoặc tinh chỉnh mô hình**: fine-tuning LLM với dữ liệu đặc thù.
3. **Tích hợp vào logic ứng dụng**: kết nối mô hình với các thành phần hệ thống.
4. **Triển khai qua pipeline CI/CD**: Docker + SageMaker hoặc nền tảng tương tự.
5. **Giám sát**: Prometheus, Datadog – độ trễ, tỷ lệ lỗi, chất lượng đầu ra.
6. **Phản hồi và cập nhật**: thu thập phản hồi, đánh giá lại mô hình, lặp lại.

Công cụ hỗ trợ: **DVC** (Data Version Control – như Git cho dữ liệu và mô

Ví dụ: Chatbot tư vấn khóa học với RAG (1/2)

Bài toán: Xây dựng chatbot trả lời câu hỏi về chương trình đào tạo dựa trên đề cương môn học (PDF). **Công nghệ sử dụng**:

- `langchain` – quản lý pipeline LLM.
- `streamlit` – giao diện web.
- `chromadb` – vector database nhẹ, chạy local.
- `openai` – gọi GPT-4o + Embeddings API.
- `pypdf` – đọc file PDF đề cương.

Pipeline RAG:

1. Đọc tất cả PDF $\rightarrow$ chia thành chunks (`chunk_size=800`, `chunk_overlap=100`).
2. Tạo embedding $\rightarrow$ lưu vào ChromaDB.
3. Khi có câu hỏi: embedding câu hỏi $\rightarrow$ tìm k đoạn liên quan nhất $\rightarrow$ gửi cùng prompt đến GPT-4o.

Ví dụ: Chatbot tư vấn khóa học với RAG (2/2)

Code lõi của pipeline RAG:

```
# --- Khoi tao hoac load vector store ---
@st.cache_resource
def get_vector_store():
    """Load hoac tao moi vector store tu PDF trong thu muc data."""
    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

    if os.path.exists(CHROMA_DIR) and os.listdir(CHROMA_DIR):
        # Load vector store da co
        return Chroma(
            persist_directory=CHROMA_DIR,
            embedding_function=embeddings,
        )

    # Kiem tra thu muc du lieu
    if not os.path.exists(DATA_DIR):
        os.makedirs(DATA_DIR)
        st.warning(f"Thu muc '{DATA_DIR}' trong. Vui long dat file PDF de
        ↪  cuong vao thu muc nay.")
        return None

    # Load PDF
    loader = PyPDFDirectoryLoader(DATA_DIR)
```

Tổng kết thực hành – Bạn đã làm được gì?

Sau 2 tiết thực hành – bạn có thể:

1. **Chọn đúng công cụ** cho từng giai đoạn: Gradio/Streamlit cho prototype, Flask/Django cho production.
2. **Thực hiện quy trình 10 bước** xây dựng chatbot GenAI – từ use case đến giám sát và cải tiến.
3. **Xây dựng pipeline RAG** hoàn chỉnh: PDF $\rightarrow$ chunks $\rightarrow$ embeddings $\rightarrow$ vector database $\rightarrow$ truy xuất $\rightarrow$ sinh câu trả lời.
4. **Sử dụng AI coding assistants** (Copilot, Q Developer, Tabnine) để tăng tốc phát triển.
5. **Hiểu quy trình GenAI Ops**: từ prototype đến production qua CI/CD, Docker, giám sát và cải tiến liên tục.

Bài tập về nhà

Tổng kết chương

Những điểm cần ghi nhớ

- **UI/UX là cầu nối trung tâm** giữa sức mạnh kỹ thuật của AI và nhu cầu người dùng – không chỉ là "vỏ bọc" mà quyết định thành công của sản phẩm AI tạo sinh.
- **Hệ sinh thái công cụ chia ba cấp độ**: tạo mẫu nhanh (Gradio, Streamlit), sản xuất (Flask, Django + React/Vue), và tích hợp front-end AI (LangChain.js). Chọn đúng công cụ cho từng giai đoạn.
- **Kiến trúc chatbot GenAI 4 tầng**: Tương tác – Xử lý nghiệp vụ – Lưu trữ – Bảo mật. Kết hợp với Well-Architected Framework 6 trụ cột để thiết kế hệ thống toàn diện.
- **AI trong UI/UX là con dao hai lưỡi**: tăng tốc phát triển (Google: 25%+ code mới do AI sinh) nhưng đi kèm sai lệch, chi phí inference liên tục, hallucination và rủi ro đạo đức. AI là "companion", không phải "replacer".